

Banking Management System Using Core Java - Complete Project Submission Report

Abstract

Complete mini-project for Banking Management System developed using Core Java and Object-Oriented Programming.

Introduction

This project automates banking operations such as account creation, deposits, withdrawals, fund transfers, etc.

Problem Statement

Manual banking operations are time-consuming and error-prone. A software-based system improves accuracy and efficiency.

Objectives

1. Manage customer accounts.
2. Perform deposits and withdrawals.
3. Transfer funds.
4. Maintain transaction history.
5. Demonstrate Core Java concepts.

Scope

The system supports retail banking operations in a console-based environment.

Software Requirements Specification

Functional Requirements:

- Create Account
- Deposit
- Withdraw
- Transfer Funds
- Balance Inquiry
- Transaction History

Non-Functional Requirements:

- Reliability
- Performance
- Security
- Maintainability

Detailed User Stories

- US-01: Customer creates account.
- US-02: Customer deposits money.
- US-03: Customer withdraws money.
- US-04: Customer transfers funds.
- US-05: Customer checks balance.
- US-06: Customer reviews transaction history.

Detailed Use Case UC-01 Create Account

Actor: Customer

Preconditions: Customer not registered.

Main Flow:

1. Select Create Account.
2. Enter customer details.
3. Validate details.
4. Generate account number.
5. Create account.
6. Display success.

Alternative Flow:

Invalid details entered.

Postcondition:

Account successfully created.

Detailed Use Case UC-02 Deposit Money

Actor: Customer

Main Flow:

1. Enter account number.
2. Enter amount.
3. Validate account.
4. Update balance.
5. Record transaction.
6. Display confirmation.

Detailed Use Case UC-03 Withdraw Money

Actor: Customer

Main Flow:

1. Enter account number.
2. Enter withdrawal amount.
3. Verify balance.
4. Deduct amount.
5. Save transaction.
6. Show confirmation.

Detailed Use Case UC-04 Transfer Funds

1. Enter source account.
2. Enter destination account.
3. Enter transfer amount.
4. Validate accounts.
5. Debit source.
6. Credit destination.
7. Record transaction.

Detailed Use Case UC-05 Balance Inquiry

1. Enter account number.
2. Validate account.
3. Retrieve balance.
4. Display balance.

Detailed Use Case UC-06 Transaction History

1. Enter account number.
2. Retrieve transactions.
3. Display complete history.

UML Use Case Diagram

```
Customer
|-- Create Account
|-- Deposit
|-- Withdraw
|-- Transfer Funds
|-- Balance Inquiry
|-- Transaction History
```

UML Class Diagram

```
BankAccount
+ accountNumber
+ customerName
+ balance
+ deposit()
+ withdraw()
```

```
Transaction
+ transactionId
+ amount
+ date
```

```
BankSystem
+ createAccount()
+ searchAccount()
+ transferFunds()
```

UML Sequence Diagram

```
Customer -> BankSystem
BankSystem -> Account
Account -> BankSystem
BankSystem -> Customer
```

UML Activity Diagram

```
Start
Select Operation
Process Request
Display Result
End
```

UML State Diagram

```
Created -> Active -> Transaction Processing -> Closed
```

Component Diagram

```
UI Layer
Business Layer
Data Layer
```

Deployment Diagram

```
Client System
Java Application
Local Storage
```

ER Diagram

```
CUSTOMER --- ACCOUNT --- TRANSACTION
```

DFD Level 0

```
Customer -> Banking System -> Reports
```

DFD Level 1

```
Account Management
Transaction Management
Balance Management
```

Database Design

```
ACCOUNT(AccountNo, Name, Balance)
```

```
TRANSACTION(TransactionId, AccountNo,
Type, Amount, Date)
```

Testing

```
TC01 Create Account
TC02 Deposit
TC03 Withdraw
```

TC04 Transfer
TC05 Balance Inquiry
TC06 Transaction History

Sample Java Classes

BankAccount.java
Transaction.java
BankSystem.java
Main.java

Sample Source Code Structure

```
public class BankAccount {  
    private int accountNo;  
    private String name;  
    private double balance;  
}
```

Advantages

Easy to use
Fast processing
Reduced manual work
Accurate calculations

Limitations

Console-based
No database integration
No online access

Future Enhancements

JDBC
MySQL
JavaFX GUI
Internet Banking
Authentication

Viva Questions

1. What is OOP?
2. What is encapsulation?
3. What is inheritance?
4. Why use collections?
5. What is exception handling?
6. What is polymorphism?
7. Explain UML.
8. Explain use cases.
9. Explain DFD.
10. Explain ER diagram.

Conclusion

The Banking Management System successfully demonstrates Core Java, OOP, collections, exception handling.